



VIT[®]
BHOPAL
www.vitbhopal.ac.in

LABORATORY RECORD

CSA4028 Natural Language Processing

C14+E11+E12, BL2025260500521,

Winter Semester:2025-2026

Name of the student: **Vijay Rajesh R**

Register Number: **23BAI10917**

B.Tech Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

Programme Output (POs):

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own

work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PSO & PEO: Link

<https://vitbhopal.ac.in/wp-content/uploads/2021/12/B.Tech-AI-ML-PO-PEO-PSO.pdf>

Index

Exp. No.	Name of the Experiment	Page Number
1.	Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	1
2.	Morphological Analysis – Python Implementation	6
3.	Generation different N-Gram models (unigram, bigram, trigram)	15
4.		
5.		
6.		
7.		

Exp No: 1	Preprocessing of text
30.01.2025	

Aim:

To preprocessing the given text and perform the following operations (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)

Objective:

1. To familiarize with preprocessing of text
2. To understand the preprocessing techniques using NLTK libraries

Simulation Tool:

Python and NLTK Libraries

Theory:

Tokenization: Text data needs to be broken down into smaller units, such as words or phrases, for analysis. Tokenization divides text into meaningful units, facilitating subsequent processing steps like feature extraction.

Stopwords: Stopwords are common words like “the,” “is,” and “and” that often occur frequently but convey little semantic meaning.

Removing stopwords can improve the efficiency of text analysis by reducing noise.

Feature Extraction: Preprocessing can involve extracting features from text, such as word frequencies, n-grams, or word embeddings, which are essential for building machine learning models.

Dimensionality Reduction: Text data often has a high dimensionality due to the presence of a large vocabulary. Preprocessing techniques like term frequency-inverse document frequency (TF-IDF) or dimensionality reduction methods can help.

Procedure:

- Removing punctuations like . , ! \$ () * % @
- Removing URLs
- Removing Stop words
- Lower casing
- Tokenization
- Stemming
- Lemmatization

Output: (*Simulation Screen Shots*)

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
```

```
nltk.download('punkt')
nltk.download('stopwords')
```

```
raw_docs = [

''' The history of Tamil Nadu is a vast timeline of cultural continuity,
beginning with deep prehistoric roots. Archaeological findings at
Attirampakkam suggest that hominin populations existed in the region as
far back as 1.5 million years ago, long before modern Homo sapiens
arrived. By the Mesolithic and Neolithic periods, humans in the region
transitioned from nomadic hunter-gatherers using microlithic tools to
settled communities that practiced agriculture and animal husbandry.'''

'''The Iron Age in Tamil Nadu is particularly noted for its megalithic
burial sites, such as those found in Adichanallur, which reveal a
sophisticated society that used iron tools, practiced elaborate funerary
rites involving urns, and potentially used an early form of writing.'''

'''As the region entered the early historic period, often referred to as
the Sangam era, it was defined by the reign of the "Three Crowned
Kings": the Cheras, Cholas, and Pandyas. These dynasties presided over a
landscape of shifting borders and constant rivalry, yet they fostered a
golden age of Tamil literature and maritime trade. Their influence was
recorded in the edicts of the Mauryan Emperor Ashoka and the
inscriptions of King Kharavela of Kalinga, indicating their sovereignty
remained intact despite the rise of northern empires. '''

'''This era saw the development of a complex social hierarchy involving
local chieftains known as Velirs and the coexistence of diverse
religious traditions, including Vedic practices alongside
Jainism and Buddhism.'''
```

```
nlk.download('punkt')
nlk.download('stopwords')

[22] ✓ 0.0s

... [nlk_data] Downloading package punkt to
[nltk_data] C:\Users\vijay\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\vijay\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!

... True
```

```
Tokens      : ['the', 'history', 'of', 'tamil', 'nadu', 'is', 'a', 'vast', 'timeline', 'of']
Stop Words   : {'but', 'theirs', "i've", 'which', 'y', "it'll", 'what', 'down', 'now', 'a',
filtered_tokens : ['history', 'tamil', 'nadu', 'vast', 'timeline', 'cultural', 'continuity', '']
```

```
#We reduce the features to 2 componenets for easy
svd = TruncatedSVD(n_components=2)
reduced_data = svd.fit_transform(tfidf_matrix)
reduced_data

26] ✓ 0.0s

... c:\Users\vijay\AppData\Local\Programs\Python\Python312\Lib\s
self.explained_variance_ratio_ = exp_var / full_var

... array([[1.]])
```


Result:

The preprocessing of given text is performed using the following techniques tokenization, filtration, script validation, stop word removal and stemming

References:

1. <https://www.geeksforgeeks.org/nlp/natural-language-processing-overview/>
2. <https://www.freecodecamp.org/news/natural-language-processing-techniques-for-beginners/>
3. https://www.tutorialspoint.com/natural_language_processing/index.htm

Aim:

To design and implement various **Morphological Analyzers** using rule-based systems, Finite State Automata (FSA), and algorithmic stemmers to identify, decompose, and transform English words into their constituent morphemes (stems and affixes).

Objective:

- **Rule-Based Morphological Analysis**
 - Stem and Suffix Separation
 - Inflectional vs. Derivational Analysis
- **Finite State Morphology**
 - Finite State Automata (FSA) for Nouns and Verbs
 - Orthographic Spelling Rules (e.g., *y* to *ies*)
- **Verb Inflectional Analysis**
 - Tense Modeling (Present, Past, Participle)
- **Algorithmic Stemming**
 - Porter Stemmer Implementation and Heuristics
- **NLP Preprocessing**
 - Lemmatization and Base Form Retrieval
 - Morphological Normalization for POS Tagging

Simulation Tool:

Python and NLTK Libraries

Theory:

Program:

Output: *(Simulation Screen Shots)*

```
    return stem, suffix

# 3. Check if word ends with "s"
elif word.endswith("s"):
    stem = word[:-1] # Remove Last 1 letter
    suffix = "s"
    return stem, suffix

# If no suffix is found
else:
    return word, "No suffix"

# --- How to use-----
my_word = "eating"
stem, suffix = split_word(my_word)

print('23BAI10917 - Vijay Rajesh R')
print("Word:", my_word)
print("Stem:", stem)
print("Suffix:", suffix)
```

5] ✓ 0.0s

```
23BAI10917 - Vijay Rajesh R
Word: eating
Stem: eat
Suffix: ing
```

```

def check_inflected_or_derived(word):
    ...# 1. Lists of endings
    ...inflections = ["ing", "ed", "es", "s"]
    ...derivations = ["ness", "ly", "ment", "able", "ion"]

    ...# 2. Check for Derivational first
    ...for suffix in derivations:
    ...    ...if word.endswith(suffix):
    ...        ...base = word[:-len(suffix)]
    ...        ...return base, "Derived"

    ...# 3. Check for Inflectional
    ...for suffix in inflections:
    ...    ...if word.endswith(suffix):
    ...        ...base = word[:-len(suffix)]
    ...        ...return base, "Inflected"

    ...return word, "Base Form"

# Test it
print(check_inflected_or_derived("happiness")) # Derived (happy + ness)
print(check_inflected_or_derived("walking")) # Inflected (walk + ing)

```

✓ 0.0s

```

('happi', 'Derived')
('walk', 'Inflected')

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS JUPYTER

C:\Users\vijay\OneDrive\Desktop\clsdump\NLP> git

```
def get_plural_base(word):  
    # Rule 1: Ends in 'ies' (like babies -> baby)  
    if word.endswith("ies"):  
        base = word[:-3] + "y"  
        return base  
  
    # Rule 2: Ends in 'es' (like boxes -> box)  
    elif word.endswith("es"):  
        base = word[:-2]  
        return base  
  
    # Rule 3: Ends in 's' (like cats -> cat)  
    elif word.endswith("s"):  
        base = word[:-1]  
        return base  
  
    return word  
  
# Test it  
print("babies ->", get_plural_base("babies"))  
print("boxes ->", get_plural_base("boxes"))  
print("cats ->", get_plural_base("cats"))
```

✓ 0.0s

```
babies -> baby  
boxes -> box  
cats -> cat
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS JUPYTER

C:\Users\vijay\OneDrive\Desktop\clsdump\NLP> git

-> Present tense

-> Past tense

-> Present participle

```
def analyze_verb(word):  
    # Rule for Present Participle  
    if word.endswith("ing"):  
        return word[:-3], "Present Participle"  
  
    # Rule for Past Tense  
    elif word.endswith("ed"):  
        return word[:-2], "Past Tense"  
  
    # Rule for Present Tense (Third Person)  
    elif word.endswith("s"):  
        # Check for 'es' first (like 'goes' or 'fixes')  
        if word.endswith("es"):  
            return word[:-2], "Present Tense"  
        return word[:-1], "Present Tense"  
  
    return word, "Present Tense (Base)"  
  
# Test it  
print(analyze_verb("running")) # run, Present Participle  
print(analyze_verb("played")) # play, Past Tense  
print(analyze_verb("walks")) # walk, Present Tense
```

38] ✓ 0.0s

```
.. ('runn', 'Present Participle')  
   ('play', 'Past Tense')  
   ('walk', 'Present Tense')
```

1. Write a program to perform stemming using Porter Stemmer

```
> ✓  
def pos_preprocess(word):  
    # Only focus on inflectional suffixes (grammar changes)  
    inflections = ["ing", "ed", "es", "s"]  
  
    # Try to find an inflectional suffix  
    for suffix in inflections:  
        if word.endswith(suffix):  
            base_form = word[:-len(suffix)]  
            return base_form  
  
    return word # If no suffix, return as is  
  
# Example  
words_to_clean = ["running", "fixed", "dogs"]  
cleaned = [pos_preprocess(w) for w in words_to_clean]  
  
print("Original:", words_to_clean)  
print("Base Forms:", cleaned)
```

[39] ✓ 0.0s

```
... Original: ['running', 'fixed', 'dogs']  
Base Forms: ['runn', 'fix', 'dog']
```

2. Write a program to perform morphological analysis for

- > POS tagging preprocessing by:
- > Removing inflectional suffixes
- > Returning the base form of words

```
def pos_preprocess(word):  
    # Only focus on inflectional suffixes (grammar changes)  
    inflections = ["ing", "ed", "es", "s"]  
  
    # Try to find an inflectional suffix  
    for suffix in inflections:  
        if word.endswith(suffix):  
            base_form = word[:-len(suffix)]  
            return base_form  
  
    return word # If no suffix, return as is  
  
# Example  
words_to_clean = ["running", "fixed", "dogs"]  
cleaned = [pos_preprocess(w) for w in words_to_clean]  
  
print("Original:", words_to_clean)  
print("Base Forms:", cleaned)
```

[40] ✓ 0.0s

```
... Original: ['running', 'fixed', 'dogs']  
Base Forms: ['runn', 'fix', 'dog']
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS JUPYTER

PS C:\Users\vijay\OneDrive\Desktop\clsdump\NLP> git

3. Design and implement a morphological analyzer using Finite State Automaton (FSA)

```
def fsa_morphology(word, category):
    # category can be "noun" or "verb"
    state = "START"

    if category == "noun":
        # State: Check for plural 's'
        if word.endswith("s"):
            state = "PLURAL"
            base = word[:-1]
            if word.endswith("ies"): # Special case
                base = word[:-3] + "y"
        else:
            state = "SINGULAR"
            base = word

    elif category == "verb":
        # State: Check for tense
        if word.endswith("ing"):
            state = "PRESENT_PARTICIPLE"
            base = word[:-3]
        elif word.endswith("ed"):
            state = "PAST_TENSE"
            base = word[:-2]
        else:
            state = "BASE_VERB"
            base = word

    return base, state

# Testing the FSA Logic
print("FSA Noun (babies):", fsa_morphology("babies", "noun"))
print("FSA Verb (played):", fsa_morphology("played", "verb"))
```

✓ 0.0s

```
FSA Noun (babies): ('baby', 'PLURAL')
FSA Verb (played): ('play', 'PAST_TENSE')
```

Result:

Morphological analysis involves the systematic decomposition of words into their constituent morphemes, specifically isolating the stem from its inflectional or derivational suffixes. By employing Finite State Automata (FSA) to handle complex orthographic transitions—such as the transformation of "baby" to "babies"—and utilizing the Porter Stemmer for heuristic-based suffix stripping, these systems successfully normalize various word forms into a standardized base. This process is critical for Natural Language Processing (NLP) pipelines, as it facilitates accurate Part-of-Speech (POS) tagging and reduces vocabulary dimensionality by mapping diverse linguistic variations to a single, unified lemma.

References:

1. <https://www.geeksforgeeks.org/nlp/natural-language-processing-overview/>
2. <https://www.freecodecamp.org/news/natural-language-processing-techniques-for-beginners/>
3. https://www.tutorialspoint.com/natural_language_processing/index.htm

Exp No: 3	Generate different N-Gram models (unigram, bigram, trigram, etc.)
13.02.2026	

Aim:

To implement and generate various N-Gram models (Unigram, Bigram, and Trigram) using Python to understand word sequences and linguistic context.

Objective:

- To tokenize a custom corpus of 10 sentences.
- To apply the concept of a sliding window to extract contiguous sequences of items.
- To utilize the Natural Language Toolkit (NLTK) for efficient text processing.

Simulation Tool:

- **Language:** Python 3.x
- **Library:** NLTK (Natural Language Toolkit)
- **Environment:** IDLE, Jupyter Notebook, or VS Code.

Theory:

An **N-gram** is a contiguous sequence of n items from a given sample of text or speech.

- **Unigram (n=1):** Each word is treated independently.
- **Bigram (n=2):** Each token is dependent on the previous one.
- **Trigram (n=3):** Each token is dependent on the two preceding tokens.

Program:

```
import nltk
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

# Download necessary resources
nltk.download('punkt')
```

```

# Corpus: 10 Call of Duty Dialogue Sentences
corpus = [
    "Fifty thousand people used to live here, now it is a ghost town.",
    "History is written by the victor.",
    "The healthy human mind does not wake up thinking this is its last day.",
    "We get our hands dirty so the world stays clean.",
    "The numbers, Mason! What do they mean?",
    "Right. What the hell kind of name is Soap, eh?",
    "Bravo Six, going dark.",
    "All it takes is the will of a single man.",
    "My name is Viktor Reznov and I will have my revenge!",
    "The enemy of my enemy is my friend."
]

def generate_ngrams(text, n):
    tokens = word_tokenize(text.lower())
    return list(ngrams(tokens, n))

# Iterating through the corpus
for i, sentence in enumerate(corpus, 1):
    print(f"Sentence {i}: {sentence}")
    print(f"  Unigrams: {generate_ngrams(sentence, 1)}")
    print(f"  Bigrams: {generate_ngrams(sentence, 2)}")
    print(f"  Trigrams: {generate_ngrams(sentence, 3)}")
    print("-" * 50)

```

Output:

```

Sentence 1: Fifty thousand people used to live here, now it is a ghost town.
Unigram: [('fifty',), ('thousand',), ('people',), ('used',), ('to',), ('live',), ('here',), ('.',), ('now',), ('it',), ('is')]
Bigram: [('fifty', 'thousand'), ('thousand', 'people'), ('people', 'used'), ('used', 'to'), ('to', 'live'), ('live', 'here')]
Trigram: [('fifty', 'thousand', 'people'), ('thousand', 'people', 'used'), ('people', 'used', 'to'), ('used', 'to', 'live')]
-----
Sentence 2: History is written by the victor.
Unigram: [('history',), ('is',), ('written',), ('by',), ('the',), ('victor',), ('.',)]
Bigram: [('history', 'is'), ('is', 'written'), ('written', 'by'), ('by', 'the'), ('the', 'victor'), ('victor', '.')]
Trigram: [('history', 'is', 'written'), ('is', 'written', 'by'), ('written', 'by', 'the'), ('by', 'the', 'victor'), ('the',
-----
Sentence 3: The healthy human mind does not wake up thinking this is its last day.
Unigram: [('the',), ('healthy',), ('human',), ('mind',), ('does',), ('not',), ('wake',), ('up',), ('thinking',), ('this',),
Bigram: [('the', 'healthy'), ('healthy', 'human'), ('human', 'mind'), ('mind', 'does'), ('does', 'not'), ('not', 'wake'), ('
Trigram: [('the', 'healthy', 'human'), ('healthy', 'human', 'mind'), ('human', 'mind', 'does'), ('mind', 'does', 'not'), ('
-----
Sentence 4: We get our hands dirty so the world stays clean.
Unigram: [('we',), ('get',), ('our',), ('hands',), ('dirty',), ('so',), ('the',), ('world',), ('stays',), ('clean',), ('.',
Bigram: [('we', 'get'), ('get', 'our'), ('our', 'hands'), ('hands', 'dirty'), ('dirty', 'so'), ('so', 'the'), ('the', 'world')]
Trigram: [('we', 'get', 'our'), ('get', 'our', 'hands'), ('our', 'hands', 'dirty'), ('hands', 'dirty', 'so'), ('dirty', 'so
-----
Sentence 5: The numbers, Mason! What do they mean?
Unigram: [('the',), ('numbers',), ('.',), ('mason',), ('!',), ('what',), ('do',), ('they',), ('mean',), ('?',)]
Bigram: [('the', 'numbers'), ('numbers', '.'), ('.', 'mason'), ('mason', '!'), ('!', 'what'), ('what', 'do'), ('do', 'they')]
Trigram: [('the', 'numbers', '.'), ('numbers', '.', 'mason'), ('.', 'mason', '!'), ('mason', '!', 'what'), ('!', 'what', 'd
-----
...
Unigram: [('the',), ('enemy',), ('of',), ('my',), ('enemy',), ('is',), ('my',), ('friend',), ('.',)]
Bigram: [('the', 'enemy'), ('enemy', 'of'), ('of', 'my'), ('my', 'enemy'), ('enemy', 'is'), ('is', 'my'), ('my', 'friend')]
Trigram: [('the', 'enemy', 'of'), ('enemy', 'of', 'my'), ('of', 'my', 'enemy'), ('my', 'enemy', 'is'), ('enemy', 'is', 'my')]

```

Result:

The N-Gram models were successfully generated for the 10-sentence *Call of Duty* corpus. The program demonstrates how increasing the value of n provides more contextual information at the cost of increasing the sparsity of the data.

References:

1. <https://www.geeksforgeeks.org/nlp/natural-language-processing-overview/>
2. <https://www.freecodecamp.org/news/natural-language-processing-techniques-for-beginners/>
3. https://www.tutorialspoint.com/natural_language_processing/index.htm

Exp No:	
	

Aim:

...

Objective:

-

Simulation Tool:

....

Theory:

.....

Program:

...

Output:

Result:

References:

1. <https://www.geeksforgeeks.org/nlp/natural-language-processing-overview/>
2. <https://www.freecodecamp.org/news/natural-language-processing-techniques-for-beginners/>
3. https://www.tutorialspoint.com/natural_language_processing/index.htm